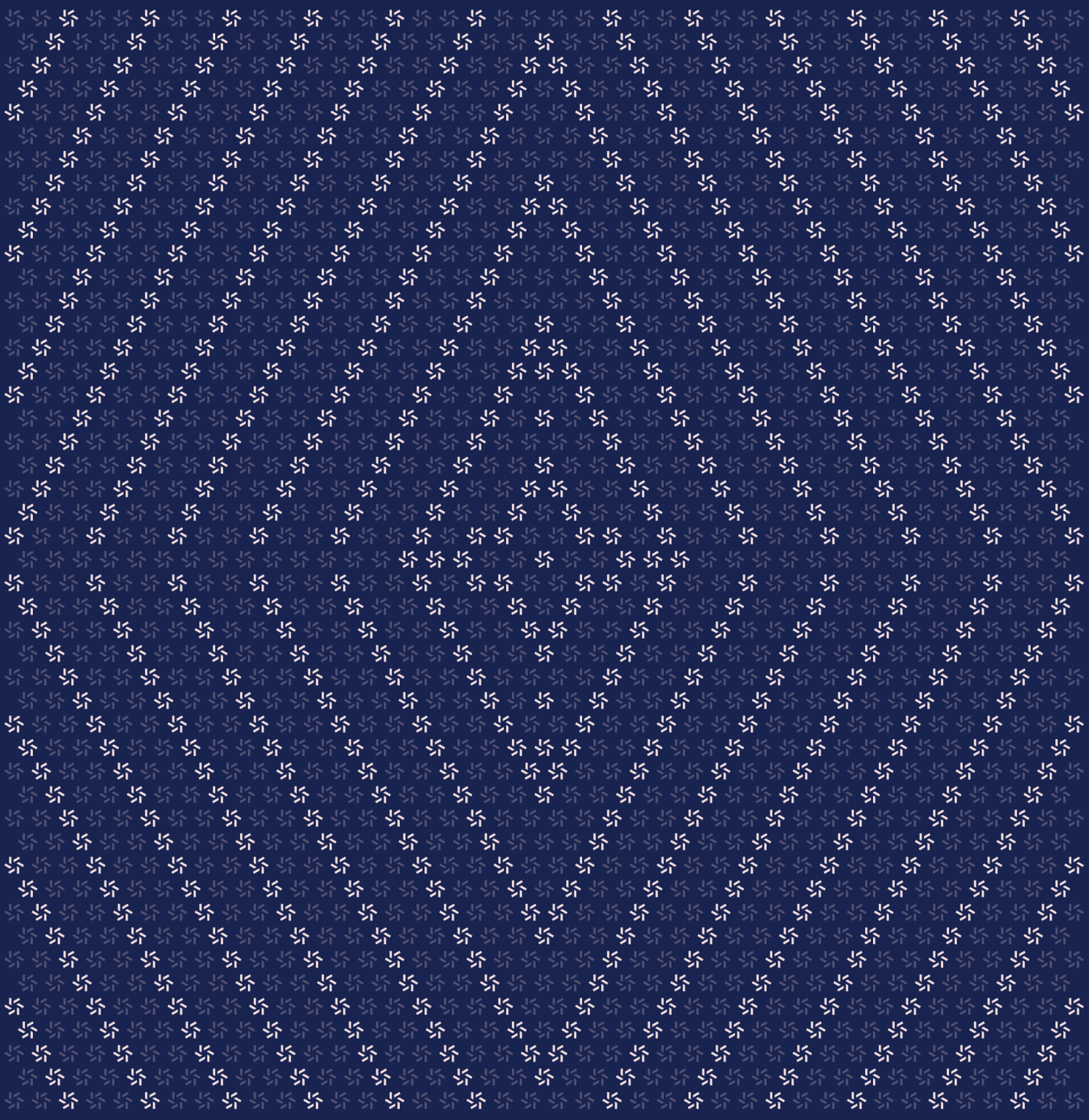
Zellic

Prepared for
Jake Goh
Superstate, Inc.

Prepared by
Jie Ding
Hojung Han
Zellic

June 23, 2026

Superstate EVM
Smart Contract Security Assessment



Contents

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the #1 CTF (competitive hacking) team ↗ worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io ↗ and follow @zellic_io ↗ on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io ↗.

1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Superstate, Inc. from June 17th to June 19th, 2026. During this engagement, Zellic reviewed Superstate EVM's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Are there any vulnerabilities that could let tokens be minted, burned, bridged, or redeemed without authorization or let tokens be moved to or from non-allowlisted addresses?
- Does the v5.1 to v1.2.0 migration preserve all security-critical state, or can it silently drop a control such as the accounting pause?
- Are the global pause and the accounting pause properly enforced across every minting and burning path?
- Is the in-place AllowlistV4_2 upgrade storage-safe, and are its permission and signature-based authorization checks free from bypass?
- Is token issuance priced correctly in the subscription and DIP purchase flows, with no way to mint at a manipulated price?
- Does the DIP v1.1.0 storage-preserving upgrade keep the existing storage layout intact?
- Is the Scalable UI multiplier strictly display-only, and do the `setName` and `setSymbol` setters behave safely?

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

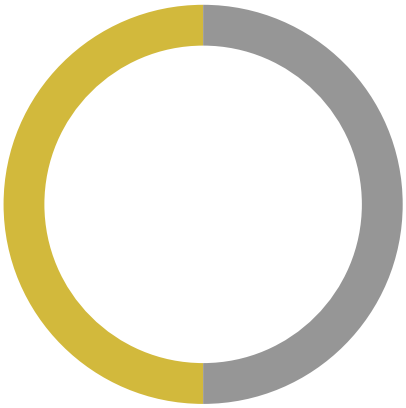
Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

During our assessment on the scoped Superstate EVM contracts, we discovered two findings. No critical issues were found. One finding was of medium impact and the other finding was informational in nature.

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	1
Low	0
Informational	1



2. Introduction

2.1. About Superstate EVM

Superstate, Inc. contributed the following description of Superstate EVM:

> Superstate is a tokenized financial instruments platform managing on-chain Treasury Bills (USTB), Crypto Carry Fund (USCC), and equity tokens. This audit covers five contract systems: a unified Allowlist for KYC/AML compliance, two upgradeable token implementations (FundToken and EquityToken) sharing a modular component architecture, an on-chain Redemption system with oracle-priced USDC payouts (idle and yield-bearing variants), and the DIP (Direct Issuance Protocol) for discounted equity issuance via Pyth oracle pricing. All contracts are upgradeable (UUPS or initializer-based) and target Solidity 0.8.28.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the contracts.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

2.3. Scope

The engagement involved a review of the following targets:

Superstate EVM Contracts

Type	Solidity
Platform	EVM-compatible
Target	superstate-evm-audit
Repository	https://github.com/superstateinc/superstate-evm-audit/ ↗
Version	`1b6d2c631d540182568b6bff871f16f428b9d98f`

Programs	`allowlist/src/v4.2/AllowlistV4_2.sol`
	`token/src/base/ERC20MetadataSettable.sol`
	`token/src/base/SuperstateTokenCore.sol`
	`token/src/components/AccountingPausable.sol`
	`token/src/components/Bridgeable.sol`
	`token/src/components/Dippable.sol`
	`token/src/components/Redeemable.sol`
	`token/src/components/Scalable.sol`
	`token/src/components/Subscribable.sol`
	`token/src/equity/v1.4.0/EquityTokenV1_4_0.sol`
	`token/src/fund/v1.2.0/FundTokenV1_2_0.sol`
	`dip/src/Dip.sol`

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 4.5 person-days. The assessment was conducted by two consultants over the course of 2.5 calendar days.

Contact Information

The following project manager was associated with the engagement:

Pedro Moura
Engagement Manager
pedro@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Jie Ding
Engineer
jie@zellic.io ↗

Hojung Han
Engineer
hojung@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

June 17, 2026	Start of primary review period
June 19, 2026	End of primary review period

3. Detailed Findings

3.1. Migration does not carry over the v5.1 accounting-pause state

Target	FundTokenV1_2_0		
Category	Business Logic	**Severity**	Medium
Likelihood	Low	**Impact**	Medium

Description

In v5.1, `accountingPaused` lives at contiguous storage slot 754. During migration, `_initializeV1_2_0()` reads only slots 755, 756, 758, and 759:

```
uint256 maxDelay  = _getUint256FromSlot(755);
address oracle    = _getAddressFromSlot(756);
address allowlist = _getAddressFromSlot(758);
address redemption= _getAddressFromSlot(759);
```

Slot 754 is never read, and `__AccountingPausable_init()` sets the new namespaced flag to `false`. So if the v5.1 token was `accountingPaused == true` before the upgrade, v1.2.0 silently clears it. The new flag gates mint, burn, bridge, redeem, subscribe, and book-entry burn, while the global `pause()` (slot 101) is preserved — so the two controls diverge after migration.

Impact

An upgrade run while accounting-paused (e.g., during an incident) reenables all accounting-gated flows, while `paused()` still reports `true`.

Recommendations

Carry the flag over: `if (_getBoolFromSlot(754)) _setAccountingPaused(true);` (`_getBoolFromSlot` already exists, unused). Or require operators not to migrate while accounting-paused.

Only FundToken is affected. EquityToken keeps its accounting pause in the ERC-7201 namespace and never reads continuous storage.

Remediation

This issue has been acknowledged by Superstate, Inc., and a fix was implemented in commit `6216afed` ↗.

3.2. EquityToken and FundToken use different errors for allowlist denial

Target	EquityTokenV1_4_0, FundTokenV1_2_0		
Category	Code Maturity	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

EquityToken reverts `Unauthorized()` for both the owner check (line 509) and the allowlist check (line 539). FundToken distinguishes them — `Unauthorized()` for the owner (line 573) and `InsufficientPermissions()` for the allowlist (line 583).

Since `InsufficientPermissions` is declared in the shared `IAllowlistable` and is usable from EquityToken, using the same error for both is a deliberate choice. On EquityToken, callers cannot tell an owner failure from an allowlist failure.

Impact

This causes reduced error clarity; integrations handling both tokens see two errors for the same condition.

Recommendations

Confirm the uniform `Unauthorized` is intentional, or switch EquityToken's allowlist denial to `InsufficientPermissions()`.

Remediation

This issue has been acknowledged by Superstate, Inc., and a fix was implemented in commit `251c060d` ↗.

4. Threat Model

This provides a full threat model description for various functions. As time permitted, we analyzed each function in the contracts and created a written threat model for some critical functions. A threat model documents a given function's externally controllable inputs and how an attacker could leverage each input to cause harm.

Not all functions in the audit scope may have been modeled. The absence of a threat model in this section does not necessarily suggest that a function is safe.

4.1. AccountingPausable.sol

Function: `function accountingPause() external virtual`

The `accountingPause` function activates the accounting pause flag for an authorized caller. When this flag is set, all issuance and burn paths are blocked — including `mint`, `bulkMint`, `adminBurn`, `offchainRedeem`, `bridge`, `buyTheDip`, `subscribe`, and book-entry transfer-to-self. It operates independently of the global pause.

Branches and code coverage

Intended branches

- If accounting is not currently paused, it sets `accountingPaused` to `true` and emits `AccountingPaused(msg.sender)`.
  - ☑ Test coverage

Negative branches

- If the caller is not authorized, `_requireAuth()` reverts.
  - ☑ Test coverage
- If accounting is already paused, the function reverts with `AccountingIsPaused()`.
  - ☑ Test coverage

Function: `function accountingUnpause() external virtual`

The `accountingUnpause` function clears the accounting pause flag for an authorized caller. Once cleared, all issuance and burn paths are reenabled.

Branches and code coverage

Intended branches

- If accounting is currently paused, it sets `accountingPaused` to `false` and emits `AccountingUnpaused(msg.sender)`.

☑ Test coverage

Negative branches

- If the caller is not authorized, `_requireAuth()` reverts.

  ☑ Test coverage
- If accounting is not paused, the function reverts with `AccountingIsNotPaused()`.

  ☑ Test coverage

4.2. AllowlistV4_2.sol

Function: `function setAllowed(address addr, address token, bool allowed) external onlyOwner`

The `setAllowed` function allows the owner to directly set the permission flag for a given `addr` under a specific `token` scope. It delegates to `_setAllowed`, which validates the nonzero address and the current state, then updates `allowed[token][addr]` and emits `AllowedSet`.

Inputs

- `addr`
  - **Control**: Owner only.
  - **Constraints**: Must not be `address(0)`.
  - **Impact**: The target address whose permission is being granted or revoked.
- `token`
  - **Control**: Owner only.
  - **Constraints**: N/A.
  - **Impact**: Determines which token scope to update. Passing `address(0)` targets the PUBLIC scope.
- `allowed`
  - **Control**: Owner only.
  - **Constraints**: Must differ from the currently stored value — otherwise reverts.
  - **Impact**: Enables (`true`) or disables (`false`) the permission for `addr` under the given token scope.

Branches and code coverage

Intended branches

- If `addr` is valid and the new value differs from the current state, it updates

`allowed[token][addr]` and emits `AllowedSet`.

- ☑ Test coverage

Negative branches

- If the caller is not the owner, the `onlyOwner` modifier reverts with `OwnableUnauthorizedAccount`.
  - ☑ Test coverage
- If `addr == address(0)`, the function reverts with `ZeroAddress()`.
  - ☑ Test coverage
- If `allowed[token][addr]` is already set to the same value, the function reverts with `AlreadySet()`.
  - ☑ Test coverage

Function: `function setAllowedBatch(address[] calldata addresses, address token, bool allowed) external onlyOwner`

The `setAllowedBatch` function sets the permission flag for multiple addresses under the same token scope in a single transaction. It calls `_setAllowed` sequentially for each address; if any individual call fails, the entire transaction reverts.

Inputs

- `addresses`
  - **Control**: Owner only.
  - **Constraints**: No element may be `address(0)`; if any element is already set to the same state, the call reverts at that element.
  - **Impact**: The list of addresses whose permissions are updated in bulk.
- `token`
  - **Control**: Owner only.
  - **Constraints**: N/A
  - **Impact**: Determines which token scope to update for all addresses.
- `allowed`
  - **Control**: Owner only.
  - **Constraints**: For each address, must differ from its current state — otherwise reverts at that element.
  - **Impact**: Sets all addresses in the array to the same permission value for the given token scope.

Branches and code coverage

Intended branches

- If all addresses are valid and each has a different current state, it updates `allowed[token][addr]` for each and emits `AllowedSet` per entry.
  - ☑ Test coverage

Negative branches

- If the caller is not the owner, the `onlyOwner` modifier reverts with `OwnableUnauthorizedAccount`.
  - ☑ Test coverage
- If any address in the array is `address(0)`, the function reverts with `ZeroAddress()`.
  - ☑ Test coverage
- If any address in the array is already set to the same `allowed` value, the function reverts with `AlreadySet()`.
  - ☑ Test coverage

Function: `function setAllowedWithSignature(address addr, address token, bool allowed, uint256 nonce, uint256 deadline, uint8 v, bytes32 r, bytes32 s) external`

The `setAllowedWithSignature` function allows anyone to set a permission on behalf of the owner by providing a valid EIP-712 structured message signed off chain. It validates the deadline, nonce, and ECDSA signature in sequence, then consumes the nonce and calls `_setAllowed`.

Inputs

- `addr`
  - **Control**: Owner only (fixed via off-chain signature).
  - **Constraints**: Must not be `address(0)` — otherwise `_setAllowed` reverts.
  - **Impact**: The target address. Determined by the owner at signing time and cannot be changed by the caller.
- `token`
  - **Control**: Owner only (fixed via off-chain signature).
  - **Constraints**: N/A.
  - **Impact**: The token scope to update. Determined by the owner at signing time.
- `allowed`

- **Control**: Owner only (fixed via off-chain signature).
- **Constraints**: Must differ from the currently stored value — otherwise reverts.
- **Impact**: Sets the permission for `addr` under the given token scope.

- `nonce`
  - **Control**: Owner only (fixed via off-chain signature).
  - **Constraints**: Must equal `$.nonces[addr]`.
  - **Impact**: Replay protection. After consumption, `nonces[addr]` becomes `nonce + 1`.
- `deadline`
  - **Control**: Owner only (fixed via off-chain signature).
  - **Constraints**: `block.timestamp <= deadline` must hold.
  - **Impact**: Limits the validity window of the signature.
- `v`, `r`, `s`
  - **Control**: Arbitrary.
  - **Constraints**: Must constitute a valid ECDSA signature over the EIP-712 structured message by the owner.
  - **Impact**: Used for signature verification. The recovered address must match `owner()`.

Branches and code coverage

Intended branches

- If all checks pass, it increments `nonces[addr]` and calls `_setAllowed` to update the permission.
  - ☑ Test coverage

Negative branches

- If `block.timestamp > deadline`, the function reverts with `SignatureExpired()`.
  - ☑ Test coverage
- If `$.nonces[addr] != nonce`, the function reverts with `InvalidNonce()`.
  - ☑ Test coverage
- If the recovered ECDSA address differs from `owner()`, the function reverts with `InvalidSigner()`.
  - ☑ Test coverage
- If `addr == address(0)`, `_setAllowed` reverts with `ZeroAddress()`.
  - ☑ Test coverage

- If `allowed[token][addr]` is already set to the same value, `_setAllowed` reverts with `AlreadySet()`.

  ☑ Test coverage

Function call analysis

- `ECDSA.recover(digest, v, r, s)`
  - **What is controllable?** `v`, `r`, and `s` are caller-supplied; `digest` is an EIP-712 hash constructed from `addr`, `token`, `allowed`, `nonce`, and `deadline`.
  - **If the return value is controllable, how is it used and how can it go wrong?** The recovered address must match `owner()` for the permission change to proceed. Recovery failure reverts with `InvalidSigner()`.
  - **What happens if it reverts, reenters, or does other unusual control flow?** If the signature format is malformed, the ECDSA library reverts and no permission change occurs.

4.3. Bridgeable.sol

Function: `function setChainIdSupport(uint256 chainId, bool supported) external virtual`

The `setChainIdSupport` function allows an authorized caller to set whether a given chain ID is supported as a bridge destination. It delegates to `_setChainIdSupport`, which validates the duplicate-state and current–chain-ID restrictions, updates `supportedChainIds[chainId]`, and emits `SetChainIdSupport`. Chain ID 0 (book-entry) is enabled automatically during initialization.

Inputs

- `chainId`
  - **Control**: Arbitrary but constrained by `_requireAuth()`.
  - **Constraints**: Must not equal `block.chainid`. Must not already have the same `supported` state.
  - **Impact**: Determines whether tokens can be bridged to this chain ID.
- `supported`
  - **Control**: Arbitrary but constrained by `_requireAuth()`.
  - **Constraints**: Must differ from the currently stored value — otherwise reverts.
  - **Impact**: Enables (`true`) or disables (`false`) bridge support for the given chain ID.

Branches and code coverage

Intended branches

- If the `chainId` is valid and `supported` differs from the current state, it updates `supportedChainIds[chainId]` and emits `SetChainIdSupport`.
  - ☑ Test coverage

Negative branches

- If the caller is not authorized, `_requireAuth()` reverts.
  - ☑ Test coverage
- If `supported` equals the currently stored value, the function reverts with `BadArgsBridgeable()`.
  - ☑ Test coverage
- If `chainId == block.chainid`, the function reverts with `BridgeChainIdDestinationNotSupported()`.
  - ☑ Test coverage

Function: `function bridge(uint256 amount, address ethDestinationAddress, string memory otherDestinationAddress, uint256 chainId) public virtual`

The `bridge` function burns the caller's tokens to initiate a cross-chain transfer or a book-entry conversion to Superstate's off-chain ledger. After accounting pause and allowlist checks, it validates the supported chain ID and destination address exclusivity, burns the tokens via `_burn`, and emits the `Bridge` event.

Inputs

- `amount`
  - **Control**: Arbitrary.
  - **Constraints**: Must not be zero. If it exceeds the caller's balance, `_burn` reverts internally.
  - **Impact**: The number of tokens to burn.
- `ethDestinationAddress`
  - **Control**: Arbitrary.
  - **Constraints**: Must not be set simultaneously with a nonempty `otherDestinationAddress`. If `chainId == 0`, must be `address(0)`.
  - **Impact**: The destination EVM address on the target chain.
- `otherDestinationAddress`

  - **Control**: Arbitrary.
  - **Constraints**: Must not be set simultaneously with a nonzero `ethDestinationAddress`. If `chainId == 0`, must be empty.
  - **Impact**: The destination address string for non-EVM chains.
- `chainId`
  - **Control**: Arbitrary.
  - **Constraints**: The `supportedChainIds[chainId]` must be `true`.
  - **Impact**: Determines the destination chain — `0` signals a book-entry conversion.

Branches and code coverage

Intended branches

- If all checks pass, it burns `amount` from the caller via `_burn` and emits the `Bridge` event.
  - ☑ Test coverage
- If `chainId == 0` and both destination fields are empty, a book-entry burn is executed.
  - ☑ Test coverage

Negative branches

- If accounting is paused, `_requireNotAccountingPaused` reverts.
  - ☑ Test coverage
- If the caller is not on the allowlist, `_requireAllowed` reverts.
  - ☑ Test coverage
- If `amount == 0`, the function reverts with `ZeroSuperstateTokensOutBridgeable()`.
  - ☑ Test coverage
- If `ethDestinationAddress != address(0)` and `bytes(otherDestinationAddress).length != 0`, the function reverts with `TwoDestinationsInvalid()`.
  - ☑ Test coverage
- If `chainId == 0` and either `ethDestinationAddress != address(0)` or `otherDestinationAddress` is nonempty, the function reverts with `OnchainDestinationSetForBridgeToBookEntry()`.
  - ☑ Test coverage
- If `isChainIdSupported(chainId) == false`, the function reverts with `BridgeChainIdDestinationNotSupported()`.
  - ☑ Test coverage

4.4. Dip.sol

Function: `function setPriceOracle(address nextOracle) external onlyOwner`

The `setPriceOracle` function allows the owner to update the Pyth oracle contract address used for pricing all markets. It delegates to `_setPriceOracle`, which validates the nonzero address and duplicate address conditions, updates `priceOracle`, and emits `PriceOracleUpdated`. Changing the oracle takes effect immediately for all active markets.

Inputs

- `nextOracle`
  - **Control**: Owner only.
  - **Constraints**: Must not be `address(0)`. Must not equal the current `priceOracle` address.
  - **Impact**: The new Pyth oracle contract address. All subsequent `buyTheDip` and `calculateOutput` calls will use this oracle's price.

Branches and code coverage

Intended branches

- If the new oracle address is valid, it updates `priceOracle` and emits `PriceOracleUpdated(oldOracle, nextOracle)`.
  - ☐ Test coverage

Negative branches

- If the caller is not the owner, the `onlyOwner` modifier reverts with `OwnableUnauthorizedAccount`.
  - ☐ Test coverage
- If `nextOracle == address(0)`, the function reverts with `ZeroAddress()`.
  - ☑ Test coverage
- If `nextOracle` equals the current `priceOracle`, the function reverts with `SamePriceOracle()`.
  - ☑ Test coverage

Function: `function createMarket(bytes32 marketId, IERC20 paymentToken, IERC20 instrument, bytes32 oraclePriceFeedId, uint8 oracleLatencyToleranceInSeconds, Config calldata config) external onlyOwner`

The `createMarket` function allows the owner to create a new DIP direct-issuance market. After validating all parameters, it stores the market data flattened into `markets[marketId]` and sets the state to `Initialized`. Purchases are not possible in the `Initialized` state; a separate `setMarketState` call is required to transition to `Active`.

Inputs

- `marketId`
  - **Control**: Owner only.
  - **Constraints**: Must not be `bytes32(0)`. Must not already exist.
  - **Impact**: The unique identifier for the market.
- `paymentToken`
  - **Control**: Owner only.
  - **Constraints**: Must not be `address(0)`.
  - **Impact**: The ERC-20 token used for payment.
- `instrument`
  - **Control**: Owner only.
  - **Constraints**: Must not be `address(0)`.
  - **Impact**: The equity token address to be issued.
- `oraclePriceFeedId`
  - **Control**: Owner only.
  - **Constraints**: Must not be `bytes32(0)`.
  - **Impact**: The Pyth feed ID used for pricing this market.
- `oracleLatencyToleranceInSeconds`
  - **Control**: Owner only.
  - **Constraints**: Must not be zero.
  - **Impact**: Maximum acceptable oracle data staleness in seconds. Oracle data older than this value is rejected at purchase time.
- `config`
  - **Control**: Owner only.
  - **Constraints**: The `description` must be nonempty, `recipient` must not be `address(0)`, and `discountRate` must be `< BASIS_POINTS`. The `minPayment`, `totalPaymentTarget`, and `minPriceClamp` must not be zero, and the `maxPriceClamp` must be `> minPriceClamp`.
  - **Impact**: Sets all market parameters including discount rate, recipient,

minimum payment, funding target, and price clamps.

Branches and code coverage

Intended branches

- If all parameters are valid, it stores the market data in `markets[marketId]` with `state = Initialized` and `totalPaymentReceived = 0` then emits `MarketCreated(marketId, instrument, paymentToken)`.

  ☑ Test coverage

Negative branches

- If the caller is not the owner, the `onlyOwner` modifier reverts with `OwnableUnauthorizedAccount`.

  ☑ Test coverage
- If `marketId == bytes32(0)`, the function reverts with `ZeroMarketId()`.

  ☑ Test coverage
- If `markets[marketId].marketId != bytes32(0)` (market already exists), the function reverts with `AlreadyMarketId()`.

  ☑ Test coverage
- If `paymentToken == address(0)` or `instrument == address(0)`, the function reverts with `ZeroAddress()`.

  ☑ Test coverage
- If `oraclePriceFeedId == bytes32(0)`, the function reverts with `ZeroOraclePriceFeedId()`.

  ☑ Test coverage
- If `oracleLatencyToleranceInSeconds == 0`, the function reverts with `ZeroOracleLatencyTolerance()`.

  ☑ Test coverage
- If `config` validation fails, the function reverts with one of the following: `EmptyDescription()`, `ZeroRecipient()`, `InvalidDiscountRate()`, `ZeroMinPayment()`, `ZeroTotalPaymentTarget()`, `ZeroMinPriceClamp()`, or `InvalidMaxPriceClamp()`.

  ☑ Test coverage

Function: `function setMarketConfig(bytes32 marketId, Config calldata newConfig) external onlyOwner`

The `setMarketConfig` function allows the owner to update the configuration parameters (description, recipient, discount rate, minimum payment, funding target, price clamps) of an

existing market. Markets in a terminal state (`Closed` or `Cancelled`) cannot be modified.

Inputs

- `marketId`
  - **Control**: Owner only.
  - **Constraints**: `markets[marketId].marketId != marketId` reverts (market does not exist).
  - **Impact**: The ID of the market whose configuration will be updated.
- `newConfig`
  - **Control**: Owner only.
  - **Constraints**: Must pass all validations in `_validateConfig`.
  - **Impact**: Immediately overwrites all market config fields. Takes effect even for active markets, so ongoing purchases may be affected.

Branches and code coverage

Intended branches

- If the market exists and is not in a terminal state, it updates the config fields and emits `MarketConfigUpdated(marketId, newConfig)`.
  - ☑ Test coverage

Negative branches

- If the caller is not the owner, the `onlyOwner` modifier reverts with `OwnableUnauthorizedAccount`.
  - ☑ Test coverage
- If `market.marketId != marketId`, the function reverts with `InvalidMarketId()`.
  - ☑ Test coverage
- If the market state is `Cancelled` or `Closed`, the function reverts with `AlreadyMarketClosedOrCancelled()`.
  - ☑ Test coverage
- If `newConfig` validation fails, `_validateConfig` reverts.
  - ☐ Test coverage

Function: `function setMarketState(bytes32 marketId, State nextState) external onlyOwner`

The `setMarketState` function allows the owner to transition a market's state. It delegates to `_setMarketState`, which enforces the state machine rules. When leaving `Active`, `currentMarketForInstrument` is cleared. When entering `Active`, it verifies no other active market exists for the instrument before setting `currentMarketForInstrument`. At most one `Active` market is permitted per instrument.

Inputs

- `marketId`
  - **Control**: Owner only.
  - **Constraints**: `markets[marketId].marketId != marketId` reverts.
  - **Impact**: The ID of the market whose state will be changed.
- `nextState`
  - **Control**: Owner only.
  - **Constraints**: See valid state transitions below.
  - **Impact**: Changes market state. Transitioning to `Active` enables purchases; all other states block them.

Branches and code coverage

Intended branches

- For `Initialized → Active`, it sets `currentMarketForInstrument[instrument]` and enables purchases.
  - ☑ Test coverage
- For `Initialized → Paused`, it transitions state only (no side effect on `currentMarketForInstrument`).
  - ☐ Test coverage
- For `Initialized → Cancelled`, it transitions state only (no side effect on `currentMarketForInstrument`).
  - ☐ Test coverage
- For `Initialized → Closed`, it transitions state only (no side effect on `currentMarketForInstrument`).
  - ☐ Test coverage
- For `Active → Paused`, it clears `currentMarketForInstrument[instrument]` and suspends purchases.
  - ☑ Test coverage

- For `Active → Closed`, it clears `currentMarketForInstrument[instrument]` and closes the market.
  - ☐ Test coverage
- For `Active → Cancelled`, it clears `currentMarketForInstrument[instrument]` and cancels the market.
  - ☑ Test coverage
- For `Paused → Active`, it validates no duplicate active market before reenabling purchases and setting `currentMarketForInstrument`.
  - ☑ Test coverage
- For `Paused → Cancelled`, it transitions state only (no side effect on `currentMarketForInstrument`).
  - ☑ Test coverage
- For `Paused → Closed`, it transitions state only (no side effect on `currentMarketForInstrument`).
  - ☐ Test coverage

Negative branches

- If the caller is not the owner, the `onlyOwner` modifier reverts with `OwnableUnauthorizedAccount`.
  - ☑ Test coverage
- If `market.marketId != marketId`, the function reverts with `InvalidMarketId()`.
  - ☑ Test coverage
- If `prevState == nextState`, the function reverts with `InvalidStateTransition()`.
  - ☑ Test coverage
- If `nextState == State.Initialized`, the function reverts with `InvalidStateTransition()` (initialized is only set at creation time).
  - ☑ Test coverage
- If `prevState == State.Cancelled` or `prevState == State.Closed` (terminal states), the function reverts with `AlreadyMarketClosedOrCancelled()`.
  - ☑ Test coverage
- If `nextState == State.Active` and `currentMarketForInstrument[instrument] != bytes32(0)`, the function reverts with `AlreadyCurrentMarket()`.
  - ☑ Test coverage

Function: `function buyTheDip(bytes32 marketId, address buyer, uint256 paymentAmount, uint256 minOutAmount, IERC20 paymentToken) external returns (uint256 actualPaymentAmount, uint256 instrumentAmount, address recipient)`

The `buyTheDip` function is the core function called by the instrument contract (`msg.sender`) to execute a purchase in a DIP market. It verifies that the caller is the active instrument for the given market (`currentMarketForInstrument[msg.sender] == marketId`), validates the payment token, normalizes the payment amount, calculates the discounted price, and enforces slippage protection. After updating state, it automatically transitions the market to `Closed` if remaining capacity falls below `minPayment`. The actual token transfer and minting are performed by the calling instrument contract.

Inputs

- `marketId`
  - **Control**: Instrument contract only.
  - **Constraints**: `currentMarketForInstrument[msg.sender] != marketId` reverts.
  - **Impact**: The market ID in which the purchase is executed.
- `buyer`
  - **Control**: Instrument contract only.
  - **Constraints**: N/A.
  - **Impact**: The buyer address. Recorded in the `Purchase` event.
- `paymentAmount`
  - **Control**: Instrument contract only.
  - **Constraints**: If `0`, the normalized amount will be less than `market.minPayment`, causing `InsufficientPayment()`. If it exceeds the remaining capacity, a partial fill is applied.
  - **Impact**: The payment token amount to spend. The actual payment (`actualPaymentAmount`) is determined by `min` of the input and remaining capacity.
- `minOutAmount`
  - **Control**: Instrument contract only.
  - **Constraints**: `instrumentAmount < minOutAmount` reverts.
  - **Impact**: Slippage protection. The minimum number of instrument tokens the buyer must receive.
- `paymentToken`
  - **Control**: Instrument contract only.
  - **Constraints**: `market.paymentToken != paymentToken` reverts.

- **Impact**: The ERC-20 token used for payment.

Branches and code coverage

Intended branches

- If all checks pass, it updates `totalPaymentReceived`, and, if remaining capacity is below `minPayment`, it transitions the market to `Closed` (which also clears `currentMarketForInstrument[instrument]`). It returns `actualPaymentAmount`, `instrumentAmount`, and `recipient` and emits `Purchase`.
  - ☑ Test coverage
- If `totalPaymentReceived + minPayment > totalPaymentTarget`, the market is automatically transitioned to `Closed`.
  - ☑ Test coverage
- If `paymentAmount` exceeds remaining capacity, a partial fill adjusts `actualPaymentAmount` down.
  - ☑ Test coverage

Negative branches

- If `currentMarketForInstrument[msg.sender] != marketId`, the function reverts with `NotCurrentMarket()` (market is not active or the caller is not the instrument contract).
  - ☑ Test coverage
- If `market.paymentToken != paymentToken`, the function reverts with `NotPaymentToken()`.
  - ☑ Test coverage
- If `markets[marketId].marketId != marketId`, `_calculateOutput` reverts with `InvalidMarketId()`.
  - ☑ Test coverage
- If `paymentAmount == 0` or the normalized payment is less than `market.minPayment`, the function reverts with `InsufficientPayment()`.
  - ☑ Test coverage
- If `instrumentAmount < minOutAmount`, the function reverts with `InsufficientOutput(instrumentAmount, minOutAmount)`.
  - ☑ Test coverage
- If the oracle data exceeds the maximum latency, the function reverts with `PriceFeedExceedsMaxLatency()`.
  - ☑ Test coverage
- If the oracle price does not satisfy `price > 0 && expo < 0 && expo >= -255`, the function reverts with `PriceFeedInvalidOutput()`.

☑ Test coverage

- If the normalized discounted price falls outside `[minPriceClamp, maxPriceClamp]` (after normalizing to eight decimals), the function reverts with `PriceOutOfBounds()`.

  ☑ Test coverage

- If `actualPaymentAmount == 0`, the function reverts with `ZeroActualPayment()`.

  ☑ Test coverage

Function call analysis

- `IERC20Metadata(address(market.paymentToken)).decimals()` (in `buyTheDip` body)
  - **What is controllable?** `market.paymentToken` is set by the owner at market creation and cannot be changed by the caller.
  - **If the return value is controllable, how is it used and how can it go wrong?** The returned decimals are used in `changeDecimals` for payment normalization, and `IERC20Metadata.decimals()` returns `uint8`, so truncation cannot occur. A nonstandard token returning an abnormal decimals value would distort normalization.
  - **What happens if it reverts, reenters, or does other unusual control flow?** A revert causes the entire transaction to fail.
- `IERC20Metadata(address(market.instrument)).decimals()` (via `_calculateOutput`)
  - **What is controllable?** `market.instrument` is set by the owner at market creation and cannot be changed by the caller.
  - **If the return value is controllable, how is it used and how can it go wrong?** The returned decimals are used to scale `instrumentAmount`. An abnormal value would distort the output calculation.
  - **What happens if it reverts, reenters, or does other unusual control flow?** A revert causes the entire transaction to fail.
- `priceOracle.getPriceUnsafe(market.oraclePriceFeedId)` (via `_getDiscountedPrice`)
  - **What is controllable?** `market.oraclePriceFeedId` is set by the owner at market creation. The oracle address (`priceOracle`) is also owner-controlled.
  - **If the return value is controllable, how is it used and how can it go wrong?** The returned price is used to calculate `instrumentAmount`. A manipulated oracle price could cause excessive token issuance or block purchases. The circuit breaker (`[minPriceClamp, maxPriceClamp]`) acts as the primary defense, applied after normalizing the discounted price to eight decimals.
  - **What happens if it reverts, reenters, or does other unusual control flow?** An oracle revert causes the entire purchase transaction to fail, and `getPriceUnsafe` does not enforce freshness, so latency validation is performed within the contract.

4.5. Dippable.sol

Function: `function buyTheDip(bytes32 marketId, uint256 paymentAmount, uint256 minOutAmount, IERC20 paymentToken) external virtual returns (uint256 instrumentAmount)`

The `buyTheDip` function allows a caller to purchase tokens at a discounted price through a DIP market. After the accounting pause check, it delegates purchase calculation to the DIP contract, transfers the actual payment amount (`actualPaymentAmount`) from the buyer to the recipient via `safeTransferFrom`, and mints `instrumentAmount` tokens to the buyer.

Inputs

- `marketId`
  - **Control**: Arbitrary.
  - **Constraints**: Must be a valid active market ID in the DIP contract.
  - **Impact**: Determines which DIP market to purchase from.
- `paymentAmount`
  - **Control**: Arbitrary.
  - **Constraints**: Must not be zero — otherwise reverts.
  - **Impact**: The payment token amount to spend. The DIP contract compares it against remaining market capacity to determine the actual payment amount (`actualPaymentAmount`).
- `minOutAmount`
  - **Control**: Arbitrary.
  - **Constraints**: Must not be zero — otherwise reverts.
  - **Impact**: Slippage protection. The DIP contract enforces `instrumentAmount >= minOutAmount`.
- `paymentToken`
  - **Control**: Arbitrary.
  - **Constraints**: Must match the payment token configured for the market.
  - **Impact**: Specifies the ERC-20 token used for payment.

Branches and code coverage

Intended branches

- If all checks pass, it receives `(actualPaymentAmount, instrumentAmount, paymentRecipient)` from the DIP contract, transfers the payment token, and mints `instrumentAmount` to the buyer.

☑ Test coverage

Negative branches

- If accounting is paused, `_requireNotAccountingPaused` reverts.
  - ☑ Test coverage
- If `paymentAmount == 0`, the function reverts with `ZeroPaymentAmount()`.
  - ☑ Test coverage
- If `minOutAmount == 0`, the function reverts with `ZeroMinOutAmount()`.
  - ☑ Test coverage
- If `dipContract() == address(0)`, the function reverts with `DipContractNotSet()`.
  - ☑ Test coverage
- If DIP contract internal validation fails (inactive market, slippage, payment token mismatch, etc.), the DIP contract reverts.
  - ☑ Test coverage
- If `msg.sender` (buyer) is not on the allowlist, `_mint` reverts internally (EquityToken: `Unauthorized()`; FundToken: `InsufficientPermissions()`).
  - ☐ Test coverage

Function call analysis

- `IDip(dip).buyTheDip(marketId, msg.sender, paymentAmount, minOutAmount, paymentToken)`
  - **What is controllable?** The `marketId`, `paymentAmount`, `minOutAmount`, and `paymentToken` are caller inputs, and `msg.sender` is forwarded as the buyer address.
  - **If the return value is controllable, how is it used and how can it go wrong?** The returned `actualPaymentAmount` is used as the transfer amount, and `instrumentAmount` is used as the mint quantity. If the DIP contract is replaced with an untrusted address, excessive minting or fraudulent payment routing could occur.
  - **What happens if it reverts, reenters, or does other unusual control flow?** A DIP contract revert causes the entire transaction to fail. The DIP contract address is owner-controlled via `setDipContract` and must be a trusted address.
- `paymentToken.safeTransferFrom(msg.sender, paymentRecipient, actualPaymentAmount)`
  - **What is controllable?** Both `paymentToken` and `actualPaymentAmount` depend on the DIP call result, and `paymentRecipient` is also returned by the DIP contract.

- **If the return value is controllable, how is it used and how can it go wrong?** A transfer failure reverts. A malicious DIP contract could return an arbitrary `paymentRecipient`, causing the payment token to be sent to an unintended address.
- **What happens if it reverts, reenters, or does other unusual control flow?** If `safeTransferFrom` reverts, `_mint` is also not executed. Tokens with callbacks (e.g., ERC-777) may introduce reentrancy risk; at the reentry point, the DIP call has already returned and payment has been transferred, but `_mint` has not yet executed.

Function: `function setDipContract(address newDipContract) external virtual`

The `setDipContract` function allows an authorized caller to update the DIP protocol contract address. It delegates to `_setDipContract`, which validates the nonzero and duplicate address conditions, updates the `dipContract` slot, and emits `SetDipContract`.

Inputs

- `newDipContract`
  - **Control**: Arbitrary but constrained by `_requireAuth()`.
  - **Constraints**: Must not be `address(0)`. Must not equal the current address.
  - **Impact**: Changes the DIP contract address used in `buyTheDip`. Setting a malicious address could allow manipulation of `buyTheDip` return values.

Branches and code coverage

Intended branches

- If the new address is valid, it updates `dipContract` and emits `SetDipContract(oldDipContract, newDipContract)`.
  - ☑ Test coverage

Negative branches

- If the caller is not authorized, `_requireAuth()` reverts.
  - ☑ Test coverage
- If `newDipContract == address(0)`, the function reverts with `ZeroAddress()`.
  - ☑ Test coverage
- If `newDipContract == oldDipContract`, the function reverts with `SameAddress()`.
  - ☑ Test coverage

4.6. EquityTokenV1_4_0.sol

Function: `function setAllowlist(address _allowlist) external`

The `setAllowlist` function allows the owner to change the allowlist contract address used for KYC/AML compliance checks. It delegates to `_setAllowlist`, which validates the nonzero and duplicate address conditions. The change takes effect immediately, affecting all transfer and minting paths.

Inputs

- `_allowlist`
  - **Control**: Owner only.
  - **Constraints**: Must not be `address(0)`. Must not equal the current allowlist address.
  - **Impact**: Changes the contract queried by `isAllowed()`, immediately affecting all transfer and minting operations.

Branches and code coverage

Intended branches

- If the address is valid, it updates the allowlist address and emits `AllowlistUpdated`.
  - ☑ Test coverage

Negative branches

- If the caller is not the owner, `Unauthorized()` reverts.
  - ☑ Test coverage
- If `_allowlist == address(0)`, the function reverts with `ZeroAddressNotAllowed()`.
  - ☐ Test coverage
- If `_allowlist` equals the current allowlist address, the function reverts with `AlreadySet()`.
  - ☐ Test coverage

Function: `function setIsPublicInstrument(bool _isPublicInstrument) external`

The `setIsPublicInstrument` function allows the owner to toggle the instrument type between public and private. In public mode, entity-level allowlist permissions are used; in private mode, token-symbol–based allowlist permissions are used. It delegates to `_setIsPublicInstrument`.

Inputs

- `_isPublicInstrument`
  - **Control**: Owner only.
  - **Constraints**: N/A.
  - **Impact**: Changes the allowlist scope used by `isAllowed()`. While `true` uses entity-level public scope, `false` uses token-symbol (`symbol()`)-based private scope.

Branches and code coverage

Intended branches

- It updates the `isPublicInstrument` flag and emits `IsPublicInstrumentUpdated`.
  - ☑ Test coverage

Negative branches

- If the caller is not the owner, `Unauthorized()` reverts.
  - ☑ Test coverage

Function: `function transfer(address dst, uint256 amount) public override(ERC20Upgradeable, IERC20Upgradeable) returns (bool)`

The `transfer` function is the standard ERC-20 function that moves tokens from the caller to `dst`. Two execution paths exist: if `dst == address(this)`, it performs a book-entry conversion — checks the accounting pause, burns via `_burn`, and emits the `Bridge` event; otherwise, it checks the global pause and the `dst` allowlist then performs a regular transfer.

Inputs

- `dst`
  - **Control**: Arbitrary,
  - **Constraints**: If `dst != address(this)`, must be on the allowlist — `msg.sender` must also be on the allowlist.
  - **Impact**: The token recipient. If `address(this)`, activates the book-entry burn path.
- `amount`
  - **Control**: Arbitrary,
  - **Constraints**: If it exceeds the caller's balance, `_transfer` or `_burn` reverts internally.

- **Impact**: The number of tokens to transfer or burn.

Branches and code coverage

Intended branches

- If `dst == address(this)`, checks accounting pause, burns via `_burn(msg.sender, amount)`, and emits `Bridge(msg.sender, msg.sender, amount, address(0), "", 0)`.
  - ☑ Test coverage
- If `dst != address(this)`, checks global pause, checks `dst` allowlist, and then performs `_transfer(msg.sender, dst, amount)`.
  - ☑ Test coverage

Negative branches

- If `msg.sender` is not on the allowlist, the function reverts with `Unauthorized()`.
  - ☑ Test coverage
- If `dst != address(this)` and `dst` is not on the allowlist, the function reverts with `Unauthorized()`.
  - ☑ Test coverage
- If `dst == address(this)` and accounting is paused, the function reverts with `AccountingIsPaused()`.
  - ☑ Test coverage
- If `dst != address(this)` and the contract is globally paused, the function reverts.
  - ☑ Test coverage

Function: `function transferFrom(address src, address dst, uint256 amount) public override(ERC20Upgradeable, IERC20Upgradeable) returns (bool)`

The `transferFrom` function is the standard ERC-20 function that moves tokens from `src` to `dst` by consuming the caller's allowance. If `dst == address(this)`, it takes the book-entry burn path — checks accounting pause, spends the allowance, calls `_burn(src, amount)`, and emits the `Bridge` event. Otherwise, it checks the global pause and `dst` allowlist, spends the allowance, and then calls `_transfer(src, dst, amount)`. The allowance is always consumed regardless of the destination.

Inputs

- `src`
  - **Control**: Arbitrary.
  - **Constraints**: Must be on the allowlist.
  - **Impact**: The address from which tokens are debited.
- `dst`
  - **Control**: Arbitrary.
  - **Constraints**: If `dst != address(this)`, must be on the allowlist.
  - **Impact**: The recipient address. If `address(this)`, the book-entry burn path is taken.
- `amount`
  - **Control**: Arbitrary.
  - **Constraints**: If it exceeds `src`'s balance or the caller's allowance is insufficient, the function reverts.
  - **Impact**: The number of tokens to transfer or burn.

Branches and code coverage

Intended branches

- If `dst == address(this)`, spends the allowance, calls `_burn(src, amount)`, and emits the `Bridge` event.
  - ☑ Test coverage
- If `dst != address(this)`, checks `dst` allowlist, spends the allowance, and calls `_transfer(src, dst, amount)`.
  - ☑ Test coverage

Negative branches

- If `src` is not on the allowlist, the function reverts with `Unauthorized()`.
  - ☑ Test coverage
- If `dst != address(this)` and `dst` is not on the allowlist, the function reverts with `Unauthorized()`.
  - ☑ Test coverage
- If `dst == address(this)` and accounting is paused, the function reverts with `AccountingIsPaused()`.
  - ☑ Test coverage
- If `dst != address(this)` and the contract is globally paused, the function reverts.

☑ Test coverage

- If the caller's allowance is insufficient, `_spendAllowance` reverts.

☑ Test coverage

Function: `function mint(address dst, uint256 amount) external`

The `mint` function allows the owner to issue new tokens to a specific address. After `_requireAuth` and `_requireNotAccountingPaused` checks, it calls the internal `_mint`, which additionally verifies that `dst` is on the allowlist.

Inputs

- `dst`
  - **Control**: Owner only.
  - **Constraints**: Must be on the allowlist.
  - **Impact**: The address that receives the minted tokens.
- `amount`
  - **Control**: Owner only.
  - **Constraints**: N/A.
  - **Impact**: The number of tokens to mint. Increases `totalSupply`.

Branches and code coverage

Intended branches

- If `dst` is on the allowlist and accounting is active, mints the tokens and emits `Mint(msg.sender, dst, amount)`.

☑ Test coverage

Negative branches

- If the caller is not the owner, `Unauthorized()` reverts.

☑ Test coverage

- If accounting is paused, the function reverts with `AccountingIsPaused()`.

☑ Test coverage

- If `dst` is not on the allowlist, `_mint` reverts with `Unauthorized()`.

☑ Test coverage

Function: `function bulkMint(address[] calldata dsts, uint256[] calldata amounts) external`

The `bulkMint` function allows the owner to mint tokens to multiple addresses in a single transaction. After validating that the arrays have equal lengths and are nonempty, it calls the internal `_mint` for each address. If any single mint fails, the entire transaction reverts atomically.

Inputs

- `dsts`
  - **Control**: Owner only.
  - **Constraints**: Must have the same length as `amounts` and must not be empty. Each address must be on the allowlist.
  - **Impact**: The array of recipient addresses.
- `amounts`
  - **Control**: Owner only.
  - **Constraints**: Must have the same length as `dsts` and must not be empty.
  - **Impact**: The array of token amounts to mint per recipient.

Branches and code coverage

Intended branches

- If the arrays are valid and all addresses are on the allowlist, it mints tokens to each and emits `Mint` per entry.
  - ☑ Test coverage

Negative branches

- If the caller is not the owner, `Unauthorized()` reverts.
  - ☑ Test coverage
- If accounting is paused, the function reverts with `AccountingIsPaused()`.
  - ☑ Test coverage
- If `dsts.length != amounts.length` or `dsts.length == 0`, the function reverts with `InvalidArgumentLengths()`.
  - ☑ Test coverage
- If any address in the array is not on the allowlist, `_mint` reverts with `Unauthorized()`.
  - ☑ Test coverage

Function: `function adminBurn(address src, uint256 amount) external`

The `adminBurn` function allows the owner to forcibly burn tokens from any address; `src` need not be `msg.sender`, and allowlist status is not required. After the accounting pause check, it calls `_burn(src, amount)` and emits `AdminBurn`.

Inputs

- `src`
  - **Control**: Owner only.
  - **Constraints**: N/A.
  - **Impact**: The address from which tokens are burned.
- `amount`
  - **Control**: Owner only.
  - **Constraints**: N/A.
  - **Impact**: The number of tokens to burn. If `src`'s balance is insufficient, `_burn` reverts internally.

Branches and code coverage

Intended branches

- If accounting is active, burns via `_burn(src, amount)` and emits `AdminBurn(msg.sender, src, amount)`.
  - ☑ Test coverage

Negative branches

- If the caller is not the owner, `Unauthorized()` reverts.
  - ☑ Test coverage
- If accounting is paused, the function reverts with `AccountingIsPaused()`.
  - ☑ Test coverage

4.7. FundTokenV1_2_0.sol

Function: `function initializeV1_2_0(address[] calldata stablecoins, ISubscribable.StablecoinConfig[] calldata stablecoinConfigs, uint256[] calldata supportedChainIds) external reinitializer(6)`

The `initializeV1_2_0` function is the migration function for upgrading existing USTB/USCC deployments (v5.1) to v1.2.0. Only the owner can call it, and the `reinitializer(6)` modifier

prevents reexecution. Internally, it reads v5.1 storage slots 755, 756, 758, and 759 via `assembly sload` to initialize all component namespaces and then replays the caller-supplied stablecoin configs and supported chain IDs. On-chain enumeration of Solidity mappings is not possible, so this data must be provided by the caller.

Inputs

- `stablecoins`
  - **Control**: Owner only.
  - **Constraints**: Must have the same length as `stablecoinConfigs`.
  - **Impact**: The list of stablecoin addresses to configure after migration.
- `stablecoinConfigs`
  - **Control**: Owner only.
  - **Constraints**: Must have the same length as `stablecoins`. Each `fee` must not exceed 10 bps.
  - **Impact**: Sets the sweep destination and fee for each stablecoin.
- `supportedChainIds`
  - **Control**: Owner only.
  - **Constraints**: Each `chainId` must not equal `block.chainid`. Chain ID 0 is already enabled by `__Bridgeable_init()`, so including it causes `BadArgsBridgeable()`. Duplicates in the array trigger the same revert.
  - **Impact**: The list of bridge destination chain IDs to support after migration. Incorrect values will corrupt the bridge configuration.

Branches and code coverage

Intended branches

- If all parameters are valid, initializes all component namespaces and applies stablecoin configs and chain ID support in sequence.
  - ☑ Test coverage

Negative branches

- If the caller is not the owner, `_requireAuth()` reverts with `Unauthorized()`.
  - ☐ Test coverage
- If `stablecoins.length != stablecoinConfigs.length`, the function reverts with `InvalidArgumentLengths()`.
  - ☑ Test coverage
- If the migrated allowlist address is `address(0)`, the function reverts with `ZeroAddressNotAllowed()`.

- ☐ Test coverage
- If any stablecoin fee exceeds 10 bps, the function reverts with `FeeTooHigh()`.
  - ☐ Test coverage
- If any stablecoin's `sweepDestination` and `fee` are identical to the existing config, the function reverts with `BadArgsSubscribable()`.
  - ☐ Test coverage
- If any `chainId` equals `block.chainid`, the function reverts with `BridgeChainIdDestinationNotSupported()`.
  - ☐ Test coverage
- If any `chainId` is already in a supported state (`supported == true`), the function reverts with `BadArgsBridgeable()`.
  - ☐ Test coverage

Function: `function setAllowlist(address _allowlist) external`

The `setAllowlist` function allows the owner to change the allowlist contract address used for KYC/AML compliance checks. The change takes effect immediately, affecting all transfer and minting paths.

Inputs

- `_allowlist`
  - **Control**: Owner only.
  - **Constraints**: Must not be `address(0)`. Must not equal the current allowlist address.
  - **Impact**: Changes the contract queried by `isAllowed()`.

Branches and code coverage

Intended branches

- If the address is valid, it updates the allowlist address and emits `AllowlistUpdated`.
  - ☑ Test coverage

Negative branches

- If the caller is not the owner, `Unauthorized()` reverts.
  - ☑ Test coverage
- If `_allowlist == address(0)`, the function reverts with `ZeroAddressNotAllowed()`.

- ☐ Test coverage

- If `_allowlist` equals the current allowlist address, the function reverts with `AlreadySet()`.
  - ☐ Test coverage

Function: `function setIsPublicInstrument(bool _isPublicInstrument) external`

The `setIsPublicInstrument` function allows the owner to toggle the instrument type between public and private. In public mode, entity-level allowlist permissions are used; in private mode, token-symbol–based allowlist permissions are used.

Inputs

- `_isPublicInstrument`
  - **Control**: Owner only.
  - **Constraints**: N/A.
  - **Impact**: Changes the allowlist scope used by `isAllowed()`.

Branches and code coverage

Intended branches

- It updates the `isPublicInstrument` flag and emits `IsPublicInstrumentUpdated`.
  - ☑ Test coverage

Negative branches

- If the caller is not the owner, `Unauthorized()` reverts.
  - ☑ Test coverage

Function: `function transfer(address dst, uint256 amount) public override(IERC20Upgradeable, ERC20Upgradeable) returns (bool)`

The `transfer` function is the standard ERC-20 function that moves tokens from the caller to `dst`. If `dst == address(this)`, it takes the book-entry path: checks the accounting pause, burns via `_burn`, and emits `OffchainRedeem` (FundToken's book-entry conversion mechanism). If `dst != address(this)`, it checks the global pause and `dst` allowlist and then performs a regular transfer.

Inputs

- `dst`
  - **Control**: Arbitrary.
  - **Constraints**: If `dst != address(this)`, must be on the allowlist — `msg.sender` must also be on the allowlist.
  - **Impact**: The token recipient. If `address(this)`, activates the book-entry (off-chain redemption) burn path.
- `amount`
  - **Control**: Arbitrary.
  - **Constraints**: If it exceeds the caller's balance, `_transfer` or `_burn` reverts internally.
  - **Impact**: The number of tokens to transfer or burn.

Branches and code coverage

Intended branches

- If `dst == address(this)`, it checks accounting pause, burns via `_burn(msg.sender, amount)`, and emits `OffchainRedeem(msg.sender, msg.sender, amount)`.
  - ☑ Test coverage
- If `dst != address(this)`, it checks global pause, checks `dst` allowlist, and then performs `_transfer(msg.sender, dst, amount)`.
  - ☑ Test coverage

Negative branches

- If `msg.sender` is not on the allowlist, the function reverts with `InsufficientPermissions()`.
  - ☑ Test coverage
- If `dst != address(this)` and `dst` is not on the allowlist, the function reverts with `InsufficientPermissions()`.
  - ☑ Test coverage
- If `dst == address(this)` and accounting is paused, the function reverts with `AccountingIsPaused()`.
  - ☑ Test coverage
- If `dst != address(this)` and the contract is globally paused, the function reverts.
  - ☑ Test coverage

Function: `function transferFrom(address src, address dst, uint256 amount) public override(IERC20Upgradeable, ERC20Upgradeable) returns (bool)`

The `transferFrom` function is the standard ERC-20 function that moves tokens from `src` to `dst` by consuming the caller's allowance. If `dst == address(this)`, it checks the accounting pause, spends the allowance, calls `_burn(src, amount)`, and emits `OffchainRedeem`. If `dst != address(this)`, it checks the global pause and `dst` allowlist and then delegates to `ERC20Upgradeable.transferFrom`, which handles the allowance internally. The allowance is always consumed regardless of the destination.

Inputs

- `src`
  - **Control**: Arbitrary.
  - **Constraints**: Must be on the allowlist.
  - **Impact**: The address from which tokens are debited.
- `dst`
  - **Control**: Arbitrary.
  - **Constraints**: If `dst != address(this)`, must be on the allowlist.
  - **Impact**: The recipient address. If `address(this)`, the book-entry burn path is taken.
- `amount`
  - **Control**: Arbitrary.
  - **Constraints**: If it exceeds `src`'s balance or the caller's allowance is insufficient, the function reverts.
  - **Impact**: The number of tokens to transfer or burn.

Branches and code coverage

Intended branches

- If `dst == address(this)`, checks accounting pause, spends allowance via `_spendAllowance`, burns via `_burn(src, amount)`, and emits `OffchainRedeem(msg.sender, src, amount)`.
  - ☑ Test coverage
- If `dst != address(this)`, checks `dst` allowlist, then calls `ERC20Upgradeable.transferFrom({from: src, to: dst, amount: amount})`.
  - ☑ Test coverage

Negative branches

- If `src` is not on the allowlist, the function reverts with `InsufficientPermissions()`.

  ☑ Test coverage
- If `dst != address(this)` and `dst` is not on the allowlist, the function reverts with `InsufficientPermissions()`.

  ☑ Test coverage
- If `dst == address(this)` and accounting is paused, the function reverts with `AccountingIsPaused()`.

  ☑ Test coverage
- If `dst != address(this)` and the contract is globally paused, the function reverts.

  ☑ Test coverage
- If the caller's allowance is insufficient, `_spendAllowance` (book-entry path) or `ERC20Upgradeable.transferFrom` (regular path) reverts.

  ☑ Test coverage

Function: `function mint(address dst, uint256 amount) external`

The `mint` function allows the owner to issue new tokens to a specific address. After `_checkOwner` and `_requireNotAccountingPaused` checks, it calls the internal `_mint`, which additionally verifies that `dst` is on the allowlist.

Inputs

- `dst`
  - **Control**: Owner only.
  - **Constraints**: Must be on the allowlist.
  - **Impact**: The address that receives the minted tokens.
- `amount`
  - **Control**: Owner only.
  - **Constraints**: N/A.
  - **Impact**: The number of tokens to mint. Increases `totalSupply`.

Branches and code coverage

Intended branches

- If `dst` is on the allowlist and accounting is active, it mints the tokens and emits `Mint(msg.sender, dst, amount)`.

  ☑ Test coverage

Negative branches

- If the caller is not the owner, `_checkOwner()` reverts.
  - ☑ Test coverage
- If accounting is paused, the function reverts with `AccountingIsPaused()`.
  - ☑ Test coverage
- If `dst` is not on the allowlist, `_mint` reverts with `InsufficientPermissions()`.
  - ☑ Test coverage

Function: `function bulkMint(address[] calldata dsts, uint256[] calldata amounts) external`

The `bulkMint` function allows the owner to mint tokens to multiple addresses in a single transaction. After validating that the arrays have equal lengths and are nonempty, it calls the internal `_mint` for each address. If any single mint fails, the entire transaction reverts atomically.

Inputs

- `dsts`
  - **Control**: Owner only.
  - **Constraints**: Must have the same length as `amounts` and must not be empty. Each address must be on the allowlist.
  - **Impact**: The array of recipient addresses.
- `amounts`
  - **Control**: Owner only.
  - **Constraints**: Must have the same length as `dsts` and must not be empty.
  - **Impact**: The array of token amounts to mint per recipient.

Branches and code coverage

Intended branches

- If the arrays are valid and all addresses are on the allowlist, it mints tokens to each and emits `Mint` per entry.
  - ☑ Test coverage

Negative branches

- If the caller is not the owner, `Unauthorized()` reverts.
  - ☑ Test coverage

- If accounting is paused, the function reverts with `AccountingIsPaused()`.

  ☑ Test coverage
- If `dsts.length != amounts.length` or `dsts.length == 0`, the function reverts with `InvalidArgumentLengths()`.

  ☑ Test coverage
- If any address in the array is not on the allowlist, `_mint` reverts with `InsufficientPermissions()`.

  ☑ Test coverage

Function: `function adminBurn(address src, uint256 amount) external`

The `adminBurn` function allows the owner to forcibly burn tokens from any address; `src` need not be `msg.sender`, and allowlist status is not required. After the accounting pause check, it calls `_burn(src, amount)` and emits `AdminBurn`.

Inputs

- `src`
  - **Control**: Owner only.
  - **Constraints**: N/A.
  - **Impact**: The address from which tokens are burned.
- `amount`
  - **Control**: Owner only.
  - **Constraints**: N/A.
  - **Impact**: The number of tokens to burn. If `src`'s balance is insufficient, `_burn` reverts internally.

Branches and code coverage

Intended branches

- If accounting is active, it burns via `_burn(src, amount)` and emits `AdminBurn(msg.sender, src, amount)`.

  ☑ Test coverage

Negative branches

- If the caller is not the owner, `Unauthorized()` reverts.

  ☑ Test coverage

- If accounting is paused, the function reverts with `AccountingIsPaused()`.

  ☑ Test coverage

4.8. Redeemable.sol

Function: `function offchainRedeem(uint256 amount) external virtual`

The `offchainRedeem` function allows an allowlisted address to burn its own tokens to request an off-chain redemption. After the accounting pause check and allowlist check, it calls `_burn(msg.sender, amount)` and emits the `OffchainRedeem` event.

Inputs

- `amount`
  - **Control**: Arbitrary.
  - **Constraints**: If it exceeds the caller's balance, `_burn` reverts internally.
  - **Impact**: The number of tokens to burn.

Branches and code coverage

Intended branches

- If accounting is active and the caller is on the allowlist, it burns `amount` and emits `OffchainRedeem(msg.sender, msg.sender, amount)`.

  ☑ Test coverage

Negative branches

- If accounting is paused, `_requireNotAccountingPaused` reverts.

  ☑ Test coverage
- If the caller is not on the allowlist, `_requireAllowed` reverts.

  ☑ Test coverage

Function: `function setRedemptionContract(address _newRedemptionContract) external virtual`

The `setRedemptionContract` function allows an authorized caller to update the on-chain redemption contract address. It delegates to `_setRedemptionContract`, which validates that the new address differs from the current one, emits `SetRedemptionContract`, and stores the new address. Setting to `address(0)` is permitted to disable on-chain redemptions.

Inputs

- `_newRedemptionContract`
  - **Control**: Arbitrary but constrained by `_requireAuth()`.
  - **Constraints**: Must not equal the currently stored address — otherwise reverts.
  - **Impact**: Changes the redemption contract address. Setting `address(0)` disables on-chain redemptions.

Branches and code coverage

Intended branches

- If the new address differs from the current one, it emits `SetRedemptionContract(oldAddress, newAddress)` and stores the new address.
  - ☑ Test coverage

Negative branches

- If the caller is not authorized, `_requireAuth()` reverts.
  - ☑ Test coverage
- If `_newRedemptionContract` equals the current address, the function reverts with `BadArgsRedeemable()`.
  - ☑ Test coverage

4.9. Subscribable.sol

Function: `function setStablecoinConfig(address stablecoin, address newSweepDestination, uint16 newFee) external virtual`

The `setStablecoinConfig` function allows an authorized caller to add or update the configuration for a supported stablecoin. It validates that the new `sweepDestination` and `fee` differ from the current values, stores the updated config, and emits `SetStablecoinConfig`. Setting `newSweepDestination` to `address(0)` effectively disables the stablecoin for subscriptions.

Inputs

- `stablecoin`
  - **Control**: Arbitrary but constrained by `_requireAuth()`.
  - **Constraints**: N/A.
  - **Impact**: The stablecoin address whose configuration is being set.

- `newSweepDestination`
    - **Control**: Arbitrary but constrained by `_requireAuth()`.
    - **Constraints**: Must not equal the current `sweepDestination` (combined with `newFee` check).
    - **Impact**: The address to which stablecoin payments are forwarded during `subscribe`. Setting to `address(0)` disables this stablecoin.
- `newFee`
    - **Control**: Arbitrary but constrained by `_requireAuth()`.
    - **Constraints**: Must not exceed 10 bps (`FeeTooHigh`). Must not equal the current `fee` (combined with `sweepDestination` check).
    - **Impact**: The fee deducted from the subscription amount before computing the token output.

Branches and code coverage

Intended branches

- If the new config differs from the current one and `newFee` does not exceed 10 bps, it stores the updated config and emits `SetStablecoinConfig`.
    - ☑ Test coverage

Negative branches

- If the caller is not authorized, `_requireAuth()` reverts.
    - ☑ Test coverage
- If `newFee > 10 bps`, the function reverts with `FeeTooHigh()`.
    - ☑ Test coverage
- If `newSweepDestination == oldSweepDestination && newFee == oldFee`, the function reverts with `BadArgsSubscribable()`.
    - ☑ Test coverage

Function: `function subscribe(address to, uint256 inAmount, address stablecoin) external virtual`

The `subscribe` function allows `msg.sender` to pay stablecoins on behalf of `to` to receive minted fund tokens. Internally, `_subscribe` checks the global pause and accounting pause, calculates the token output amount based on oracle price, transfers the full `inAmount` to the sweep destination, and mints the computed amount to `to`. The caller and recipient may differ, allowing a third party to subscribe on someone else's behalf.

Inputs

- `to`
  - **Control**: Arbitrary.
  - **Constraints**: Must be on the allowlist (checked inside `_mint`).
  - **Impact**: The address that receives the minted tokens.
- `inAmount`
  - **Control**: Arbitrary.
  - **Constraints**: Must not be zero — otherwise reverts.
  - **Impact**: The stablecoin amount to pay. After fee deduction, the oracle NAV determines the token output.
- `stablecoin`
  - **Control**: Arbitrary.
  - **Constraints**: `supportedStablecoins[stablecoin].sweepDestination == address(0)` reverts.
  - **Impact**: The stablecoin used for payment. Its fee and sweep destination are applied.

Branches and code coverage

Intended branches

- If all checks pass, it transfers the full `inAmount` to `sweepDestination`, mints the oracle-calculated `superstateTokenOutAmount` to `to`, and then emits `Subscribe`.
  - ☑ Test coverage

Negative branches

- If `inAmount == 0`, the function reverts with `BadArgsSubscribable()`.
  - ☑ Test coverage
- If the global pause is active, `_requireNotPaused` reverts.
  - ☑ Test coverage
- If accounting is paused, `_requireNotAccountingPaused` reverts.
  - ☑ Test coverage
- If `supportedStablecoins[stablecoin].sweepDestination == address(0)`, the function reverts with `StablecoinNotSupported()`.
  - ☑ Test coverage
- If `superstateOracle() == address(0)` or `maximumOracleDelay() == 0`, the function reverts with `OnchainSubscriptionsDisabled()`.

☑ Test coverage

- If Chainlink oracle data is bad (`isBadData == true`), the function reverts with `BadChainlinkData()`.

  ☑ Test coverage

- If the computed `superstateTokenOutAmount == 0`, the function reverts with `ZeroSuperstateTokensOutSubscribable()`.

  ☑ Test coverage

- If `to` is not on the allowlist, `_mint` reverts with `InsufficientPermissions()`. Plesae note that this revert occurs after `safeTransferFrom` has already succeeded.

  ☑ Test coverage

Function call analysis

- `IERC20(stablecoin).safeTransferFrom(msg.sender, sweepDestination, inAmount)`
  - **What is controllable?** `stablecoin` and `inAmount` are caller inputs; `sweepDestination` is an owner-configured value.
  - **If the return value is controllable, how is it used and how can it go wrong?** A transfer failure reverts. A malicious ERC-20 token could trigger a reentrancy callback via `transferFrom`. At that reentry point, the stablecoin transfer has already completed but `_mint` has not yet executed; Subscribable storage is unchanged.
  - **What happens if it reverts, reenters, or does other unusual control flow?** If the transfer reverts, `_mint` is not executed.
- `AggregatorV3Interface(superstateOracle()).latestRoundData()` (via `_getChainlinkPrice`)
  - **What is controllable?** The oracle address is owner-controlled and cannot be changed by the caller.
  - **If the return value is controllable, how is it used and how can it go wrong?** If `_answer` is below `_getMinimumAcceptablePrice()` or `(block.timestamp - _chainlinkUpdatedAt)` exceeds `maximumOracleDelay()`, `isBadData = true` causes `BadChainlinkData()`. A manipulated price within valid bounds could distort `superstateTokenOutAmount`.
  - **What happens if it reverts, reenters, or does other unusual control flow?** A `latestRoundData()` revert causes the entire transaction to fail.
- `IERC20Metadata(stablecoin).decimals()` (via `_calculateSuperstateTokenOut`)
  - **What is controllable?** `stablecoin` is caller input but must pass the `supportedStablecoins` check, restricting it to owner-registered addresses.
  - **If the return value is controllable, how is it used and how can it go wrong?** The returned decimals are used for precision scaling in the token output

calculation. A nonstandard implementation returning an abnormal value would distort the minted amount.

- **What happens if it reverts, reenters, or does other unusual control flow?** A revert causes the entire transaction to fail.

- `AggregatorV3Interface(superstateOracle()).decimals()` (via `_calculateSuperstateTokenOut`)
  - **What is controllable?** The oracle address is owner-controlled.
  - **If the return value is controllable, how is it used and how can it go wrong?** The returned decimals determine `chainlinkFeedPrecision` used in the output calculation. An abnormal value would distort the minted amount.
  - **What happens if it reverts, reenters, or does other unusual control flow?** A revert causes the entire transaction to fail.

5. Assessment Results

During our assessment on the scoped Superstate EVM contracts, we discovered two findings. No critical issues were found. One finding was of medium impact and the other finding was informational in nature.

5.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Where Zellic states that a finding has been addressed or fixed in one or more specific commits, this indicates only that those commits introduce changes that, in our assessment, address the finding relative to the codebase version originally reviewed. This does not imply that Zellic reviewed other changes contained in those commits, the overall state of the codebase at those commits, or the interplay between remediation measures for different findings.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.